

Shiam Ezzo — Training Sessions

Session 1 — Why Routing? BrowserRouter, Routes, Route

Learning Objectives

By the end of this session, the team should understand:

- Why a React application needs routing.
- What `BrowserRouter`, `Routes`, and `Route` each do.
- How to map a URL to a page component.
- How the real project uses these three pieces.

1. The Problem

Mystery Room has six screens: Home, Mysteries, Mystery Details, Game, Result, and NotFound. Without routing, the browser has one address. Every screen would have to live inside one giant component, and the player could never share a link to a specific case.

Before routing, if you refreshed the page, you lost where you were. If you wanted to send a teammate a link to `/mysteries/1`, there was nothing to send — the address was always the same.

2. Familiar Idea

In plain HTML, every page has its own file and its own address. You can bookmark `/about.html` and return to it later. The browser handles navigation.

React does not work that way by default. A React app is one HTML file. But we still want the address to mean something.

3. The New Concept

React Router is a library that connects the browser's address to your components. When the address changes, the router decides which component to show.

The three core pieces:

- `BrowserRouter` — watches the browser's real address and tells everything inside it when the address changes. It is wrapped once, near the top of the app.
- `Routes` — the list of rules. Only the best matching rule is shown.
- `Route` — one rule: when the address is `path`, show `element`.

4. Tiny Example

```
<Routes>
  <Route path="/" element={<HomePage />} />
  <Route path="/about" element={<AboutPage />} />
</Routes>
```

svgsvg

When the address is `/`, React shows `HomePage`. When the address is `/about`, React shows `AboutPage`. Nothing else changes.

5. Project Application

File: `client/src/main.tsx`

```
import { BrowserRouter } from "react-router-dom";
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import App from "./App";
import "./index.css";

const rootElement = document.getElementById("root");

if (rootElement) {
  createRoot(rootElement).render(
    <StrictMode>
      <BrowserRouter>
        <App />
      </BrowserRouter>
    </StrictMode>,
  );
}
```

svgsvg

File: `client/src/App.tsx`

```
import { Route, Routes } from "react-router-dom";
import Navbar from "../components/Navbar";
import Footer from "../components/Footer";
import HomePage from "../pages/HomePage";
import NotFoundPage from "../pages/NotFoundPage";
import MysteriesPage from "../pages/MysteriesPage";
import MysteryDetailsPage from "../pages/MysteryDetailsPage";
import GamePage from "../pages/GamePage";
import ResultPage from "../pages/ResultPage";

export default function App() {
  return (
    <
      <a className="skip-link" href="#main-content">
        Skip to content
      </a>
      <Navbar />
      <div id="main-content" tabIndex={-1}>
        <Routes>
          <Route path="/" element={<HomePage />} />
          <Route path="/mysteries" element={<MysteriesPage />} />
          <Route path="/mysteries/:id" element={<MysteryDetailsPage />} />
          <Route path="/mysteries/:id/play" element={<GamePage />} />
          <Route path="/result/:id" element={<ResultPage />} />
          <Route path="*" element={<NotFoundPage />} />
        </Routes>
      </div>
      <Footer />
    </>
  );
}
```

svgsVG

6. Code Walkthrough

main.tsx — wrapping the app once

```
<BrowserRouter>
  <App />
</BrowserRouter>
```

svgsVG

What it does: Gives the whole application access to the browser's address. **Why it exists:** Every router feature needs a router above it. Without `BrowserRouter`, `Routes`, `Route`, `Link`, and `NavLink` all throw an error. **What you see:** Nothing visual. It is an invisible wrapper. **In Plain English:** "Everything inside this tag can read and change the browser's address." **Read it as:** "Wrap the app in address awareness."

`App.tsx` — the route list

```
<Routes>
  <Route path="/" element={<HomePage />} />
  ...
</Routes>
```

svgsvg

What it does: Holds the list of rules. Each `Route` maps one path to one element. **Why it exists:** Without `Routes`, the app would not know which component to show for which address. **What you see:** As the address changes, a different page appears. **In Plain English:** "When the address is this, show this." **Read it as:** "Pick the best matching rule and render its element."

The catch-all route

```
<Route path="*" element={<NotFoundPage />} />
```

svgsvg

What it does: Matches any address that no other route matched. **Why it exists:** Without it, an unknown address shows an empty screen. With it, the player sees a clear message. **What you see:** A friendly "Page not found" page. **In Plain English:** "Anything else, show NotFound." **Read it as:** "The fallback."

7. Common Mistakes

WRONG: Using `Routes` outside `BrowserRouter`.

```
<Routes> { /* throws an error */ }
```

svgsvg

React Router throws: "You should not use `Routes` outside a `Router`."

CORRECT:

```
<BrowserRouter>
  <Routes> ... </Routes>
</BrowserRouter>
```

svgsvg

WRONG: Writing the component as a child of `Route` .

```
<Route path="/community">
  <CommunityPage />
</Route>
```

svgsvg

CORRECT:

```
<Route path="/community" element={<CommunityPage />} />
```

svgsvg

WRONG: Forgetting the catch-all.

If no route matches, `Routes` shows nothing. That is what `*` fixes.

8. Important Decisions

- **Why one `BrowserRouter` ?** One app, one address bar. It is wrapped once in `main.tsx` .
- **Why `*` at the end?** A person reading the file top to bottom expects specific routes first, fallback last. The router picks the most specific match, but the order helps the reader.
- **Why `element` instead of `children`?** React Router requires `element={<...>}` for route components. Writing the component as a child throws an error.

9. Project Flow

text

```
Browser address: /mysteries/1
  |
  ▼
BrowserRouter reads the address
  |
  ▼
Routes tries each Route in order
  |
  ▼
/mysteries/:id matches
  |
  ▼
MysteryDetailsPage renders
```

svgsvg

10. Review Questions

1. What does `BrowserRouter` do?
2. What does `Routes` do?
3. What does each `Route` map?
4. Why do we need `*` ?
5. Where is `BrowserRouter` wrapped?
6. What happens if you forget `BrowserRouter` ?

11. Glossary

Term	Meaning
Router	The part that watches the address and picks the matching route.
Route	One rule: an address and what to show for it.
Page	A component that an address shows. Lives in <code>pages/</code> .
Fallback route	The route used when no other pattern matches.

12. What You Should Be Able to Explain Now

- Why a React app needs routing.
- What `BrowserRouter` , `Routes` , and `Route` each do.
- How the real `App.tsx` maps addresses to pages.
- Why the catch-all route exists.

13. Not Yet

- Dynamic segments like `:id` — that is Session 3.
- `Link` and `NavLink` — that is Session 2.
- Data fetching on each page — that is Hassan's Session 2.

Session 2 — Navigation with Link and NavLink

Learning Objectives

By the end of this session, the team should understand:

- Why a normal `<a href>` reloads the whole app.
- What `Link` does differently.
- What `NavLink` adds.

- How the project's Navbar uses both.

1. The Problem

The player needs to move between Home, Mysteries, Details, Game, and Result. Without navigation, they would have to type addresses manually. But a normal `<a href="/mysteries">` asks the server for a brand-new document, throws away the running React app, and loses all state.

2. Familiar Idea

In plain HTML, `<a href="/about.html">` takes you to another page. The browser asks the server for a new document and starts over.

That works for plain websites. For a React app, it is wasteful — every piece of state is lost, and the app restarts from zero.

3. The New Concept

- `Link` looks like a link but does something smarter: it changes the address inside the running app and lets the router swap the page. No reload, no lost state.
- `NavLink` is a `Link` that also knows whether it points at the current address. Its `className` can be a function that receives `{ isActive }`.

4. Tiny Example

```
<Link to="/mysteries">See the archive</Link>

<NavLink to="/" end>Home</NavLink>
<NavLink to="/mysteries">Mysteries</NavLink>
```

svgsvg

`Link` navigates. `NavLink` navigates and highlights itself when active.

5. Project Application

File: `client/src/components/Navbar.tsx`

```
import { Link, NavLink } from "react-router-dom";

export default function Navbar() {
  return (
    <header className="site-header">
      <Link className="brand" to="/">
        <span className="brand-mark" aria-hidden="true">
          M<span>R</span>
        </span>
        <span>
          Mystery Room<small>THE CASE ARCHIVE</small>
        </span>
      </Link>
      <nav aria-label="Main navigation">
        <NavLink to="/" end>
          Home
        </NavLink>
        <NavLink to="/mysteries">The mysteries</NavLink>
        <details className="play-guide">
          <summary>How to play</summary>
          <div className="play-guide-panel">
            <p className="eyebrow">A field guide</p>
            <h2>Follow your curiosity.</h2>
            <ol>
              <li>Open a case and read its story.</li>
              <li>Inspect the clues and choose your answer.</li>
              <li>Use a hint when you need a new perspective.</li>
              <li>Solve every stage to close the case.</li>
            </ol>
            <p>Wrong answers are part of the investigation. Take your time.</p>
          </div>
        </details>
      </nav>
    </header>
  );
}
```

svgsvg

File: client/src/pages/HomePage.tsx

```
import { Link } from "react-router-dom";

export default function HomePage() {
  return (
    <main className="home-page">
      <p className="eyebrow">Mystery Room / The Case Archive</p>
      <h1>
        Some secrets
        <br />
        deserve to be <em>found.</em>
      </h1>
      <p className="home-intro">
        A sealed room in Damascus. A castle hidden in fog. Two unfinished
        stories waiting for someone who notices the details.
      </p>
      <Link className="button" to="/mysteries">
        Explore the mysteries <span aria-hidden="true">↗</span>
      </Link>
      ...
    </main>
  );
}
```

svgsvg

File: client/src/components/MysteryCard.tsx

```
<Link to={`/mysteries/${id}`}>{title}</Link>
...
<Link className="button" to={`/mysteries/${id}`}>
  Open case <span aria-hidden="true">↗</span>
</Link>
```

svgsvg

6. Code Walkthrough

Link on the Home page

```
<Link className="button" to="/mysteries">
  Explore the mysteries
</Link>
```

svgsvg

What it does: Changes the address to `/mysteries` and lets the router swap the page. **Why it exists:** The player can enter the archive from Home. **What you see:** A styled button. Clicking it moves to the archive without reloading. **In Plain English:** "Take me to the archive." **Read it as:** "`to`" is the destination; everything inside is what the user clicks."

`NavLink` for the top navigation

```
<NavLink to="/" end>Home</NavLink>
<NavLink to="/mysteries">The mysteries</NavLink>
```

svgsvg

What it does: Renders links that know whether they are active. **Why it exists:** The player should see which page they are on. **What you see:** The active link gets the `active` class. CSS styles it. **In Plain English:** "This link knows if it points at the current page."

Why `end` on Home

```
<NavLink to="/" end>
  Home
</NavLink>
```

svgsvg

What it does: Makes the Home link active only when the whole address matches `/`. **Why it exists:** Every address starts with `/`. Without `end`, Home would be active on every page. **What you see:** Only the correct link is highlighted.

7. Common Mistakes

WRONG: Using `<a href>` inside a React Router app.

```
<a href="/mysteries">Mysteries</a>
```

svgsvg

This works, but the page reloads. All state is lost.

CORRECT:

```
<Link to="/mysteries">Mysteries</Link>
```

svgsvg

WRONG: Forgetting `end` on Home.

```
<NavLink to="/">Home</NavLink>
```

svgsvg

Now Home is active on every page. Every address starts with `/`.

WRONG: Using `href` instead of `to` on `Link`.

```
<Link href="/mysteries"> ... </Link>
```

svgsvg

`Link` uses `to`, not `href`.

8. Important Decisions

- **Why `Link` instead of `<a>`?** No reload, no lost state.
- **Why `NavLink` for the top nav?** It knows whether it points at the current address.
- **Why `end` on Home?** Because `/` is the start of every address.

9. Project Flow

text

```
Player clicks "The mysteries" (NavLink)
  |
  ▼
React Router changes the address to /mysteries
  |
  ▼
Routes matches /mysteries
  |
  ▼
MysteriesPage renders
  |
  ▼
NavLink highlights itself because isActive is true
```

svgsvg

10. Review Questions

1. What is the difference between `Link` and `<a href>`?
2. What does `NavLink` add to `Link`?
3. Why does Home need `end`?

4. Why does Community (Mysteries) not need `end` ?
5. Where is `Link` used in the project?

11. Glossary

Term	Meaning
Link	A React Router component that changes the address without reloading.
NavLink	A Link that also knows whether it points at the current address.
Active link	A navigation link that points at the current page.
<code>end</code>	A prop that makes a NavLink match only the exact address.

12. What You Should Be Able to Explain Now

- Why `<a href>` reloads and `Link` does not.
- What `NavLink` adds.
- Why Home needs `end` .
- How the project uses both.

13. Not Yet

- Dynamic routes with `:id` — Session 3.
- Programmatic navigation with `useNavigate` — Session 3.

Session 3 — Dynamic Routes and useParams

Learning Objectives

By the end of this session, the team should understand:

- Why we need dynamic routes.
- What `:id` means in a route path.
- How `useParams` reads values from the address.
- How the project uses this for two mysteries.

1. The Problem

The archive has two mysteries now, and could have twenty later. We cannot write one route per mystery. The address `/mysteries/1` and `/mysteries/2` should both show the same page, just with different data.

2. Familiar Idea

In a file system, `/home/user/file1.txt` and `/home/user/file2.txt` use the same folder structure. The name changes; the path pattern stays the same.

Routes can work the same way. A colon in the path marks a variable part.

3. The New Concept

In a route path, `:id` marks a variable segment. It matches any value in that position.

```
<Route path="/mysteries/:id" element={<MysteryDetailsPage />} />
```

svgsVG

This one route matches `/mysteries/1`, `/mysteries/2`, `/mysteries/999`, and any other value.

Inside the component, `useParams()` returns an object with the variable parts of the current address.

4. Tiny Example

```
function Greeting() {  
  const { name } = useParams();  
  return <p>Hello, {name}!</p>;  
}  
  
<Route path="/greet/:name" element={<Greeting />} />;
```

svgsVG

Visiting `/greet/Ali` shows "Hello, Ali!". The route matched, and `useParams` gave the component the name.

5. Project Application

File: `client/src/App.tsx`

```
<Route path="/mysteries/:id" element={<MysteryDetailsPage />} />  
<Route path="/mysteries/:id/play" element={<GamePage />} />  
<Route path="/result/:id" element={<ResultPage />} />
```

svgsVG

File: `client/src/pages/MysteryDetailsPage.tsx`

```
function MysteryDetailsPage() {
  const { id } = useParams();
  return id ? (
    <MysteryDetails key={id} id={id} />
  ) : (
    <Link to="/mysteries">Choose a mystery</Link>
  );
}
```

svgsvg

File: client/src/pages/GamePage.tsx

```
function GamePage() {
  const { id } = useParams();
  return id ? (
    <GameSession key={id} id={id} />
  ) : (
    <Link to="/mysteries">Choose a mystery</Link>
  );
}
```

svgsvg

File: client/src/pages/ResultPage.tsx

```
function ResultPage() {
  const { id } = useParams();
  return id ? (
    <MysteryResult key={id} id={id} />
  ) : (
    <Link to="/mysteries">Choose a mystery</Link>
  );
}
```

svgsvg

6. Code Walkthrough

The dynamic route

```
<Route path="/mysteries/:id" element={<MysteryDetailsPage />} />
```

svgsvg

What it does: Matches any address that starts with `/mysteries/` and has exactly one more segment.

Why it exists: One route for any number of mysteries. **What you see:** The same page component for different IDs. **In Plain English:** "`/mysteries/` followed by anything." **Read it as:** "Colon means variable."

Reading the ID

```
const { id } = useParams();
```

svgsvg

What it does: Reads the `id` value from the current address. **Why it exists:** The component needs to know which mystery was asked for. **What you see:** `id` is a string, always. **In Plain English:** "Give me the `id` part of the address." **Read it as:** "Destructure the params object."

The `key={id}` trick

```
<MysteryDetails key={id} id={id} />
```

svgsvg

What it does: Tells React to fully remount the inner component when `id` changes. **Why it exists:** Without `key`, React would try to reuse the same component instance, which could keep stale state.

What you see: Going from `/mysteries/1` to `/mysteries/2` shows fresh data. **In Plain English:** "Treat a different id as a different component." **Read it as:** "Key forces a reset."

The missing-id fallback

```
return id ? (
  <MysteryDetails key={id} id={id} />
) : (
  <Link to="/mysteries">Choose a mystery</Link>
);
```

svgsvg

What it does: If somehow `id` is missing, shows a link back to the archive. **Why it exists:** Defensive programming. In practice, this route always has an id. **What you see:** A fallback link.

7. Common Mistakes

WRONG: Comparing a number with a string.

```
if (mystery.id === id) // id is string, mystery.id is number
```

svgsvg

TypeScript refuses this, and it can fail silently in JavaScript.

CORRECT:

```
if (String(mystery.id) === id)
```

svgsvg

WRONG: Forgetting that `id` is a string.

```
const id = useParams().id; // string | undefined  
const number = id; // wrong, this is text
```

svgsvg

Always convert if you need a number.

WRONG: Naming mismatch.

```
<Route path="/mysteries/:mysteryId" element={<Page />} />;  
const { id } = useParams(); // undefined
```

svgsvg

The name after the colon must match the name in `useParams`.

8. Important Decisions

- **Why `:id` in the path?** One route for any mystery.
- **Why `useParams`?** The component cannot know the id otherwise.
- **Why `key={id}`?** To remount the component when the id changes.
- **Why a fallback?** Defensive coding; the id should always exist for this route.

9. Project Flow

text

```
graph TD
    A[Player clicks "Open case" on MysteryCard for id=1] --> B[Link changes address to /mysteries/1]
    B --> C[Route /mysteries/:id matches]
    C --> D[MysteryDetailsPage renders]
    D --> E[useParams() returns { id: "1" }]
    E --> F[MysteryDetails fetches mystery 1 from the server]
```

svgsvg

10. Review Questions

1. What does `:id` mean in a route path?
2. How does a component read it?
3. Why is `id` always a string?
4. Why does the project use `key={id}` ?
5. What does the fallback link guard against?

11. Glossary

Term	Meaning
Dynamic segment	A variable part of an address, written with a colon.
<code>useParams</code>	A hook that returns the variable parts of the current address.
Route parameter	A value extracted from the URL.
Remount	To fully replace a component instance with a fresh one.

12. What You Should Be Able to Explain Now

- Why we need dynamic routes.
- What `:id` and `useParams` do.
- Why `id` is always a string.
- Why the project uses `key={id}` .

13. Not Yet

- Programmatic navigation with `useNavigate` — used in `GamePage`, covered in Mulham's session.
- Data fetching for the id — that is Hassan's API layer.